

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس هوش مصنوعی نهفته تمرین کامپیوتری سوم

فرزاد جنتی	نام و نام خانوادگی
۸۱۰۱۰۳۰۹۸	شماره دانشجویی
دکتر مهدی مدرسی/دکتر صالحی	استاد
۱۴۰۴/۱۱/۲۱	تاریخ ارسال گزارش

فهرست

بخش ۱ – طراحی واحد کنترل برای Datapath عملیات ضرب-جمع (MAC).....	۴
مقدمه	۴
بخش ۱: طراحی واحد کنترل.....	۵
۱.۳ طراحی (MAC_Datapath) Datapath	۹
بخش ۲: تحلیل تأخیر و مساحت سخت‌افزاری	۱۸
سوال ۱: محاسبه و مقایسه تأخیر	۱۹
سوال ۲: محاسبه مساحت جمع‌کننده RNS	۲۴
۲/۲ محاسبه مساحت جمع‌کننده RNS	۲۴
۲/۳ محاسبه مساحت جمع‌کننده Binary	۲۵
سوال ۳: محاسبه مساحت کل مدار MAC	۲۶
جمع‌بندی نهایی	۲۹

شکل‌ها

- شکل ۱- معماری بلوک MAC TOP ۵
- شکل ۲- دیاگرام بلوکی کلی سیستم MAC ۵
- شکل ۳- نحوه اضافه کردن فایل‌های تمرین به پروژه در نرم افزار مادلسیم ۶
- شکل ۴- خروجی کامپایل فایل‌های پروژه ۷
- شکل ۵- دیاگرام FSM کنترلر دیزاین ۸
- شکل ۶- بلوک دیاگرام دیتا پث با جزئیات ۱۰
- شکل ۷- فرمت داده RNS در دیزاین ۱۱
- شکل ۸- مازول MAC_Top.v ۱۲
- شکل ۹- فایل DataMem.txt ۱۳
- شکل ۱۰- محتویات فایل DataMem.txt ۱۳
- شکل ۱۱- اضافه کردن سیگنال‌ها برای مشاهده شکل موج آنها طی فرآیند شبیه سازی ۱۴
- شکل ۱۲- شکل موج خروجی حاصل از اجرای تست بنچ در نرم افزار مدلسیم ۱۶
- شکل ۱۳- فرمت عملیات Multiply Add در سیستم اعداد باینری ۱۹
- شکل ۱۴- تاخیر عملیات MAC در سیستم باینری ۲۰
- شکل ۱۵- مقایسه دو سیستم باینری و RNS در تاخیر عملیات MAC ۲۲
- شکل ۱۶- کد مازول RNSAdder.v ۲۴
- شکل ۱۷- مقایسه تاخیر عملیات MAC در دو سیستم به تفکیک اجزا ۲۵
- شکل ۱۸- ساختار جمع کننده RNS ۲۷
- شکل ۱۹- مقایسه مساحت برای عملیات MAC در دو سیستم ۲۸

جدول‌ها

جدول ۱ - تمامی حالت ها و مقادیر FSM در کنترلر ۸

جدول ۲- سیگنال های کنترلی ۹

بخش ۱ - طراحی واحد کنترل برای Datapath عملیات ضرب-جمع (MAC)

مقدمه

هدف از این پروژه، طراحی و پیاده‌سازی یک واحد ضرب-جمع (MAC یا Multiply-Accumulate) در سیستم اعداد باقیمانده‌ای (Residue Number System - RNS) با استفاده از پیمانه‌های $\{7, 8, 9\}$ است. این پروژه شامل دو بخش اصلی می‌باشد:

۱. **بخش اول:** طراحی واحد کنترل (Controller) برای یک Datapath از پیش آماده شده و اتصال صحیح ماژول‌ها به یکدیگر

۲. **بخش دوم:** تحلیل عملکرد سیستم از نظر تأخیر و مساحت سخت‌افزاری و مقایسه با سیستم Binary

سیستم اعداد باقیمانده‌ای (RNS)

سیستم RNS یک نمایش غیرموضعی از اعداد است که در آن هر عدد صحیح X به صورت مجموعه‌ای از باقیمانده‌ها نسبت به پیمانه‌های مستقل نمایش داده می‌شود:

$$X \rightarrow (x_1, x_2, \dots, x_k) \text{ که } x_i = X \bmod m_i$$

برای پیمانه‌های $\{7, 8, 9\}$ ، محدوده دینامیکی برابر است با:

$$M = 7 \times 8 \times 9 = 504$$

مزایای RNS

- **موازی‌سازی ذاتی:** عملیات روی هر کانال مستقل از کانال‌های دیگر انجام می‌شود
- **حذف انتشار Carry:** بین کانال‌ها هیچ وابستگی وجود ندارد
- **سرعت بالاتر:** به دلیل پهنای بیت کمتر در هر کانال
- **مناسب برای DSP:** به‌ویژه برای عملیات MAC در فیلترهای دیجیتال

عملیات MAC

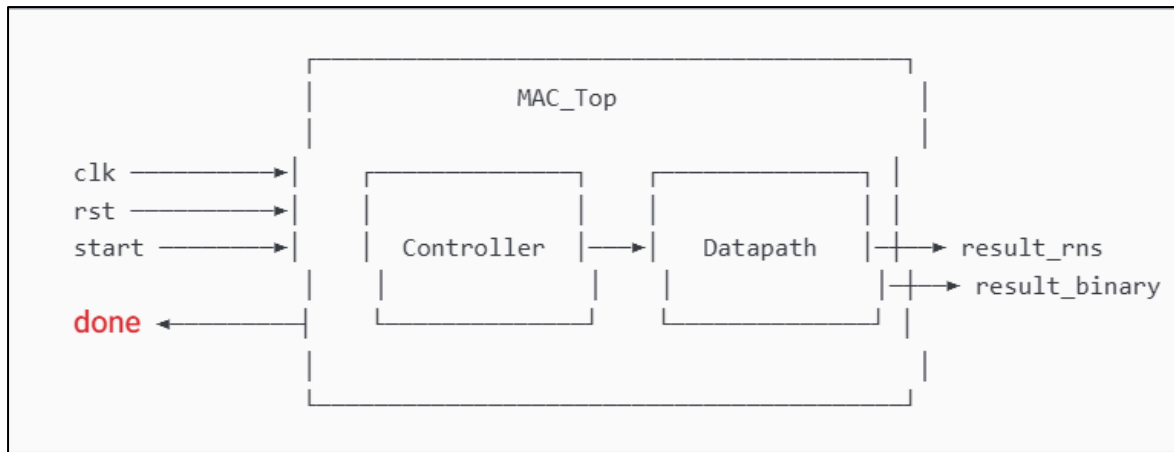
عملیات MAC که در این پروژه پیاده‌سازی شده به صورت زیر تعریف می‌شود:

$$result = \sum_{i=1}^3 a_i \cdot b_i = a_1 \cdot b_1 + a_2 \cdot b_2 + a_3 \cdot b_3$$

بخش ۱: طراحی واحد کنترل

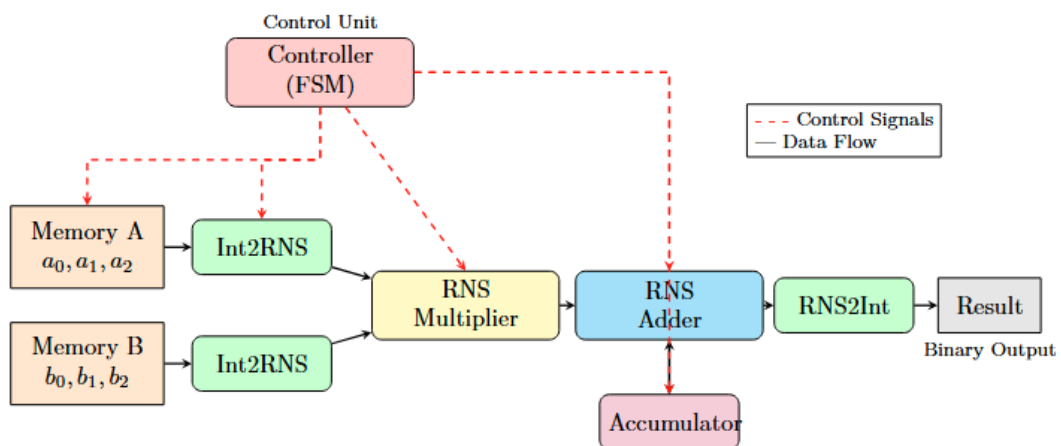
۱.۱ معماری کلی سیستم

سیستم MAC طراحی شده از سه ماژول اصلی تشکیل شده است:



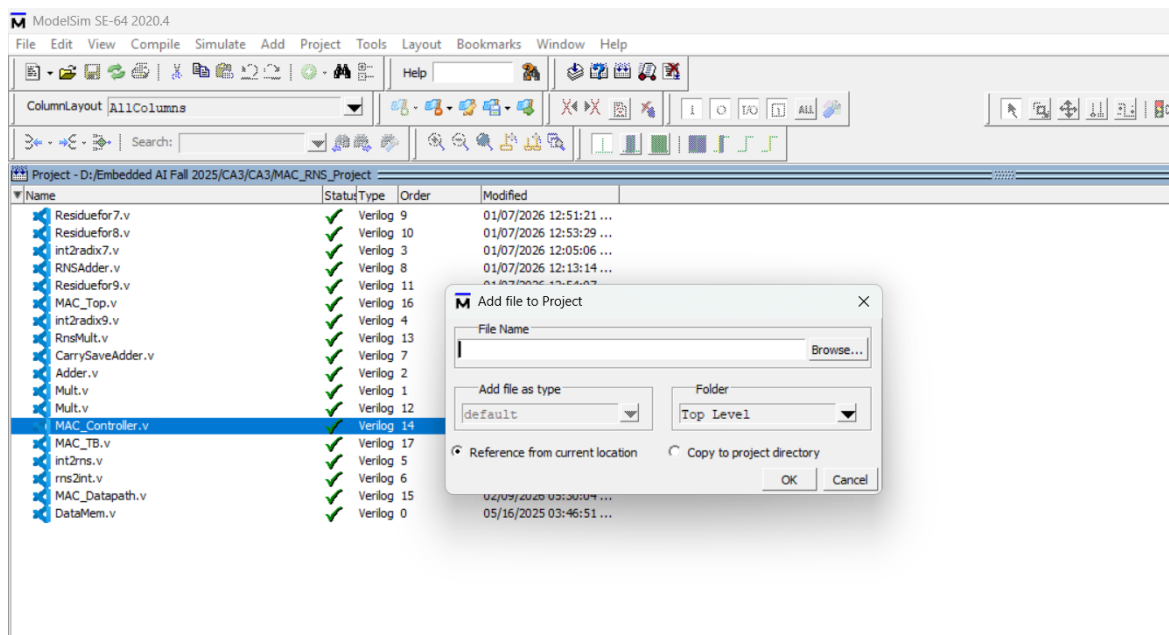
شکل ۱- معماری بلوک MAC TOP

در ادامه تصویر دقیق تری از دیتاپات و کنترلر دیزاین مون قرار گرفته است:



شکل ۲- دیاگرام بلوکی کلی سیستم MAC

قبل از اینکه به بخش تحلیل کدها و بخش کنترلر و دیتاپث پردازیم لازم به ذکر است که ما ابتدا فایل هایی که در صورت پروژه به ما داده شده است را از طریق تب *add to project* و *existing file* به ترتیب زیر به پروژه اضافه کردیم:



شکل ۳- نحوه اضافه کردن فایل های تمرین به پروژه در نرم افزار مادلسیم

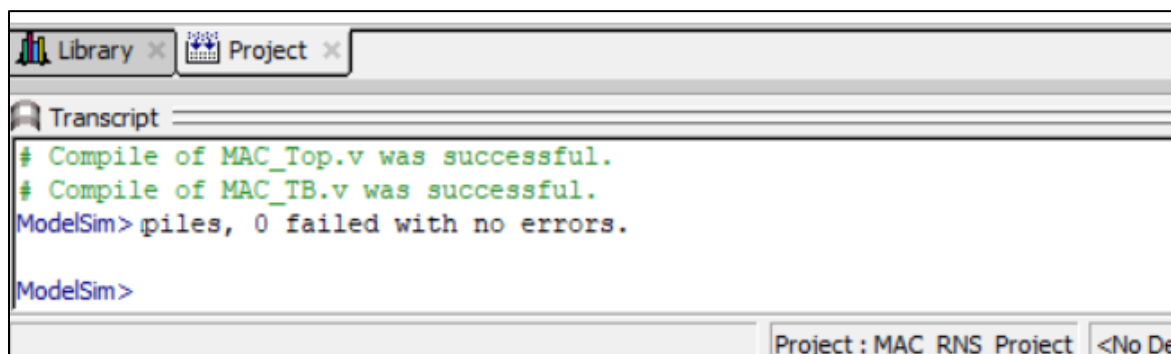
ترتیب اضافه کردن فایلها:

ترتیب	فایل	مسیر
1	DataMem.v	src/common/
2	Mult.v	src/unsigned_integer/
3	Adder.v	src/unsigned_integer/
4	int2radix7.v	src/rns/int_to_rns/
5	int2radix9.v	src/rns/int_to_rns/
6	int2rns.v	src/rns/int_to_rns/
7	rns2int.v	src/rns/rns_to_int/
8	CarrySaveAdder.v	src/rns/adder/
9	RNSAdder.v	src/rns/adder/
10	Residuefor7.v	src/rns/mult/
11	Residuefor8.v	src/rns/mult/
12	Residuefor9.v	src/rns/mult/

13	Mult.v	src/rns/mult/
14	RnsMult.v	src/rns/mult/
15	MAC_Controller.v	src/mac_unit/
16	MAC_Datapath.v	src/mac_unit/
17	MAC_Top.v	src/mac_unit/
18	MAC_TB.v	tb/

سپس تمام فایل‌های داده شده به علاوه فایل‌های اضافه شده به پروژه توسط خودمون رو کامپایل میکنیم.

نکته) برای ساختن فایل در پروژه کافست از تب *add to project* و منو *create file* رو انتخاب کنیم.



```

Library x Project x
Transcript
# Compile of MAC_Top.v was successful.
# Compile of MAC_TB.v was successful.
ModelSim> piles, 0 failed with no errors.
ModelSim>
Project : MAC_RNS_Project <No De

```

شکل ۴- خروجی کامپایل فایل‌های پروژه

همان طور که مشاهده میشود تمامی فایل‌ها به درستی و با موفقیت کامپایل شدن.

در ادامه به بررسی کنترلر دیزاین خود میپردازیم:

۱.۲ طراحی کنترلر (MAC_Controller)

۱.۲.۱ ماشین حالت متناهی (FSM)

کنترلر به صورت یک ماشین حالت Moore با ۱۵ حالت طراحی شده است:

در صفحه بعد میتونید تصویر FSM مرتبط با واحد کنترلر دیزاین داده شده رو مشاهده کنید.

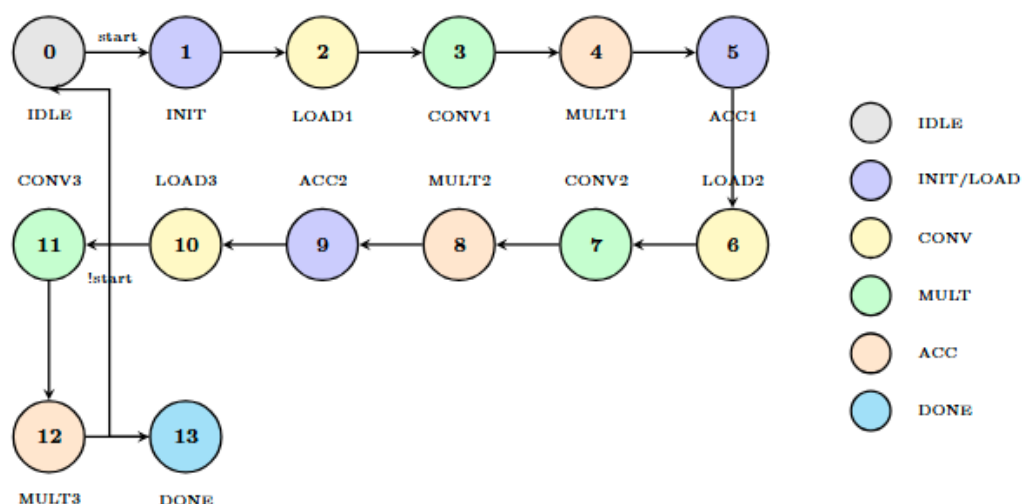


Figure 2: FSM State Diagram of MAC Controller (14 States)

شکل ۵- دیاگرام FSM کنترلر دیزاین

۱.۲.۲ جدول حالت‌ها و سیگنال‌های کنترلی

جدول ۱ - تمامی حالت‌ها و مقادیر FSM در کنترلر

حالت	کد	load_a_en	load_b_en	convert_en	mult_en	acc_en	acc_clear	operand_sel	result_valid
IDLE	0	0	0	0	0	0	0	0	0
INIT	1	0	0	0	0	0	1	0	0
LOAD1	2	1	1	0	0	0	0	0	0
CONV1	3	0	0	1	0	0	0	0	0
MULT1	4	0	0	0	1	0	0	0	0
ACC1	5	0	0	0	0	1	0	0	0
LOAD2	6	1	1	0	0	0	0	1	0
CONV2	7	0	0	1	0	0	0	0	0
MULT2	8	0	0	0	1	0	0	0	0
ACC2	9	0	0	0	0	1	0	0	0
LOAD3	10	1	1	0	0	0	0	2	0
CONV3	11	0	0	1	0	0	0	0	0

MULT3	12	0	0	0	1	0	0	0	0
ACC3	13	0	0	0	0	1	0	0	0
DONE	14	0	0	0	0	0	0	0	1

۱.۲.۳ توضیح سیگنال‌های کنترلی

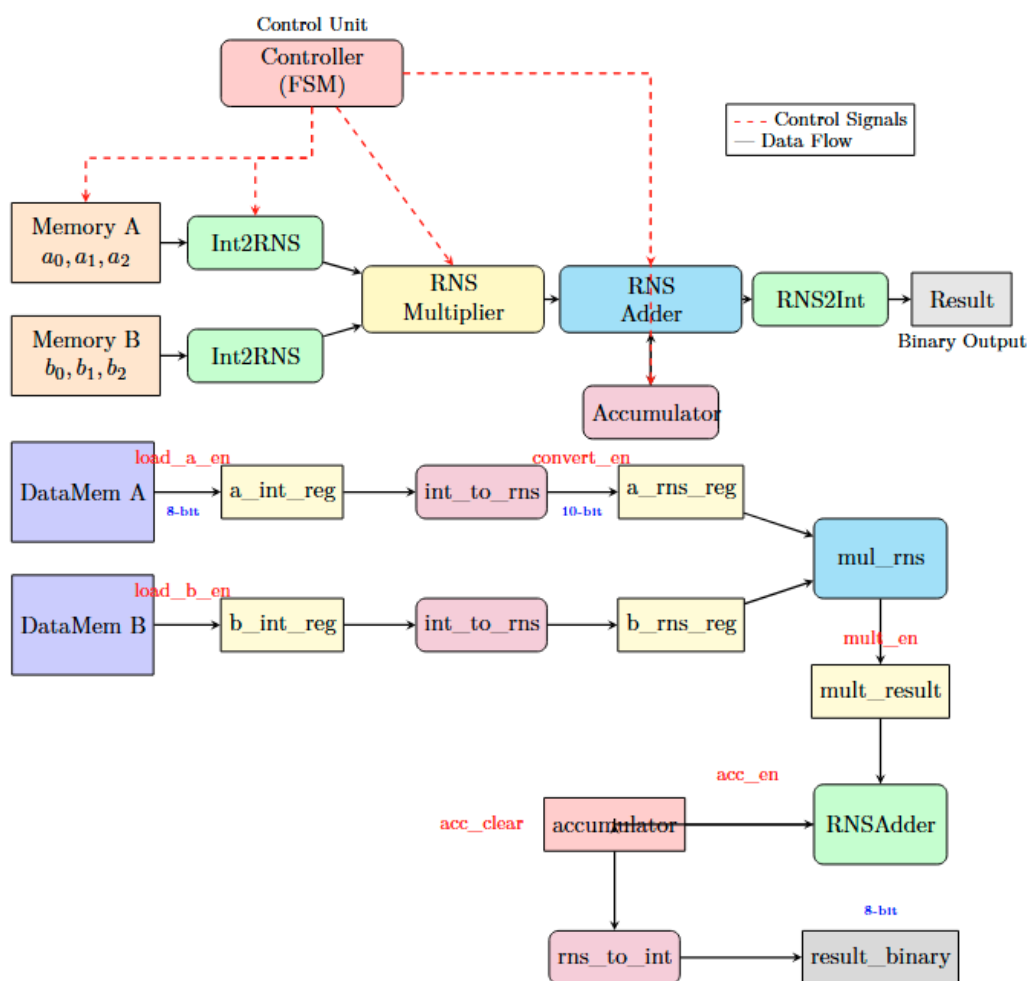
جدول ۲- سیگنال های کنترلی

سیگنال	نوع	توضیح
load_a_en	Output	فعال‌سازی بارگذاری عملوند A از حافظه
load_b_en	Output	فعال‌سازی بارگذاری عملوند B از حافظه
convert_en	Output	فعال‌سازی تبدیل از Binary به RNS
mult_en	Output	فعال‌سازی ضرب‌کننده RNS
acc_en	Output	فعال‌سازی انباشتگر (جمع با مقدار قبلی)
acc_clear	Output	پاک کردن انباشتگر (صفر کردن)
operand_sel	Output [1:0]	انتخاب شماره عملوند (۰, ۱, یا ۲)
result_valid	Output	نشان‌دهنده آماده بودن نتیجه نهایی

۱.۳ طراحی (MAC_Datapath) Datapath

۱.۳.۱ بلوک دیاگرام Datapath

در صفحه بعدی تصویر بلوک دیاگرام دیتاپات دیزاین قرار داده شده است. برای فهم بهتر کدهای کنترلر و دیتاپات طبیعتاً باید به کدهای *MAC_Datapath.v* و *MAC_Controller.v* مراجعه شود و صرفاً در اینجا به بررسی اجمالی و شهودی آن می‌پردازیم.



شکل ۶- بلوک دیاگرام دیتا پات با جزئیات

۱.۳.۲ ماژول‌های استفاده شده در Datapath

ماژول	منبع	عملکرد
DataMem	فایل اصلی	خواندن داده‌های A و B از حافظه
int_to_rns	فایل اصلی	تبدیل عدد صحیح ۸ بیتی به نمایش RNS
mul_rns	فایل اصلی	ضرب دو عدد در سیستم RNS
RNSAdder	فایل اصلی	جمع دو عدد در سیستم RNS
rns_to_int	فایل اصلی	تبدیل نمایش RNS به عدد صحیح

۱.۳.۳ فرمت داده RNS

داده‌های RNS در یک بردار ۱۰ بیتی ذخیره می‌شوند:

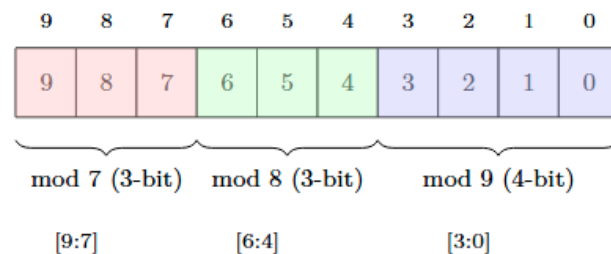


Figure 3: RNS Data Format (10-bit representation for moduli {7, 8, 9})

شکل ۷- فرمت داده RNS در دیزاین

۱.۳.۴ پارامتر MODULI

برای جمع‌کننده RNS، پیمانه‌ها به صورت زیر کدگذاری شده‌اند:

```
localparam [10:0] MODULI = 11'b111_1000_1001;
//          /   /   /
//          /   /   └── 9 (mod 9)
//          /   └── 8 (mod 8)
//          └── 7 (mod 7)
```

۱.۴ ماژول Top-Level (MAC_Top)

ماژول MAC_Top کنترلر و Datapath را به یکدیگر متصل می‌کند:

```

D:/Embedded AI Fall 2025/CA3/CA3/MAC_Top.v - Default
Ln#
2 // File: MAC_Top.v
3 // Description: Top-level module integrating Controller and Datapath
4 //=====
5
6 `timescale 1ns/1ns
7
8 module MAC_Top (
9     input wire clk,
10    input wire rst,
11    input wire start,
12
13    output wire [9:0] result_rns,
14    output wire [7:0] result_binary,
15    output wire done
16 );
17

```

شکل ۸- ماثول MAC_Top.v

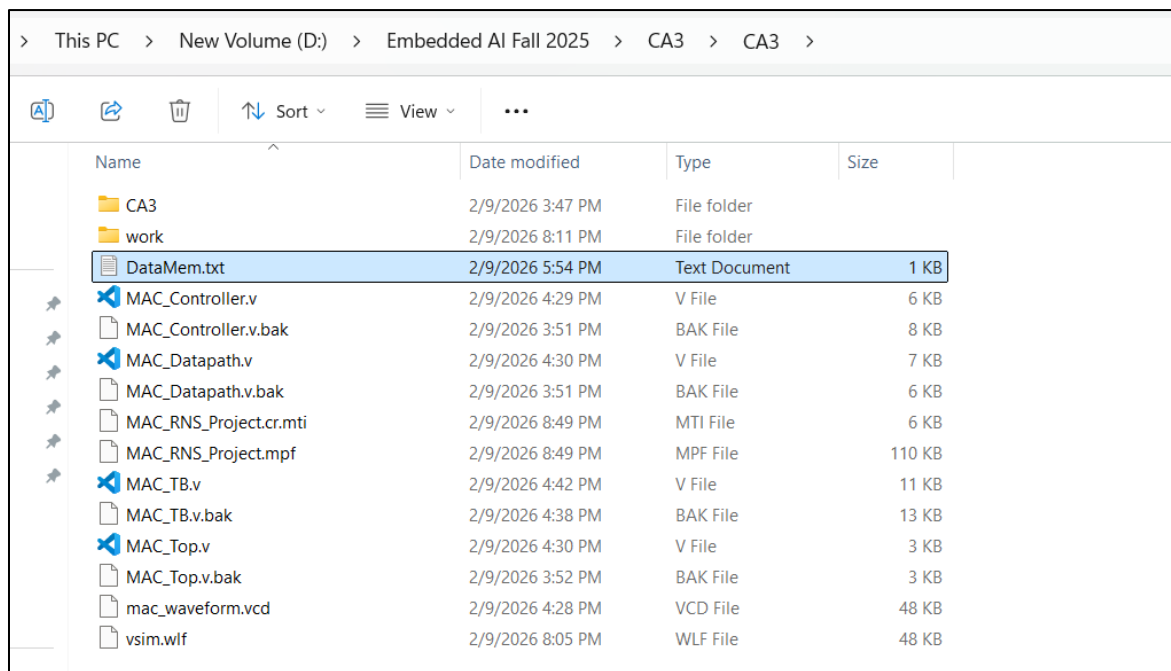
۱.۵ نتایج شبیه‌سازی

۱.۵.۱ داده‌های تست

داده‌های تست در فایل DataMem.txt:

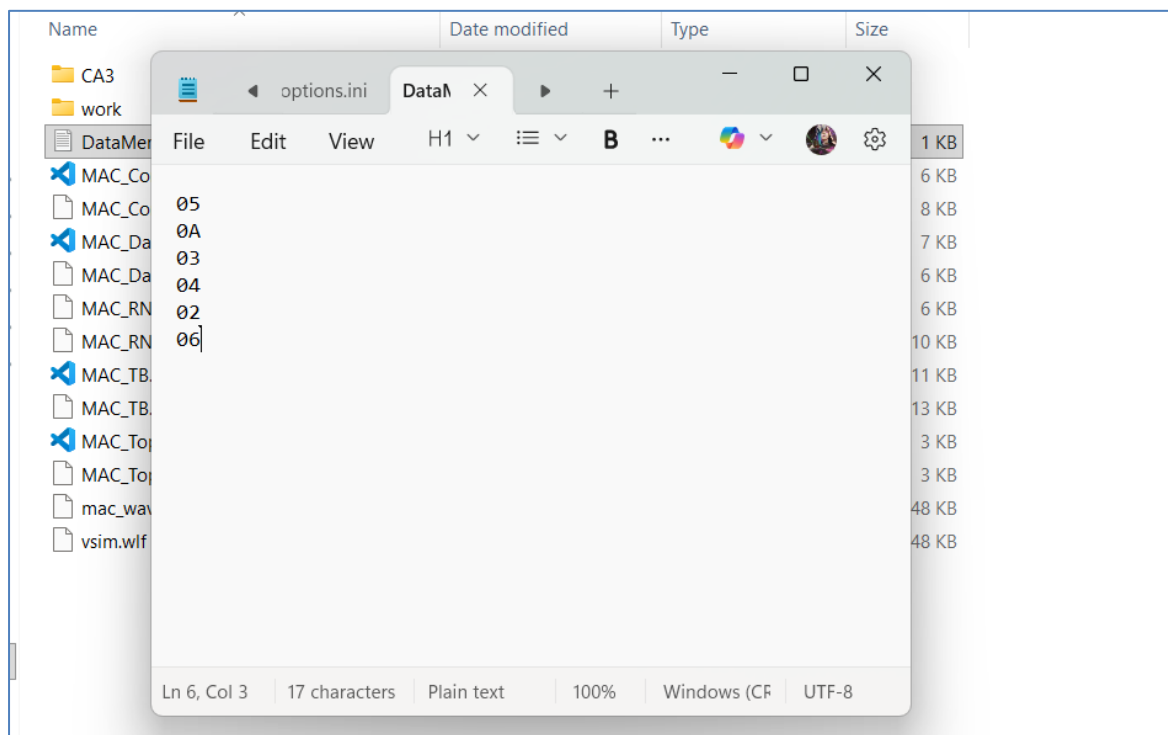
متغیر	مقدار	آدرس
a[0]	5	0
a[1]	10	1
a[2]	3	2
b[0]	4	3
b[1]	2	4
b[2]	6	5

در صفحه بعد تصویری از فایل DataMem.txt قرار داده شده است که مقادیری که قرار است در تست دیزاین مورد استفاده قرار بگیرند در آن قرار داده شده است و در کنار فایلی که شبیه سازی قرار است در آن انجام شود قرار گرفته است.



شکل ۹- فایل DataMem.txt

محتویات این فایل هم در زیر قرار داده شده است:



شکل ۱۰- محتویات فایل DataMem.txt

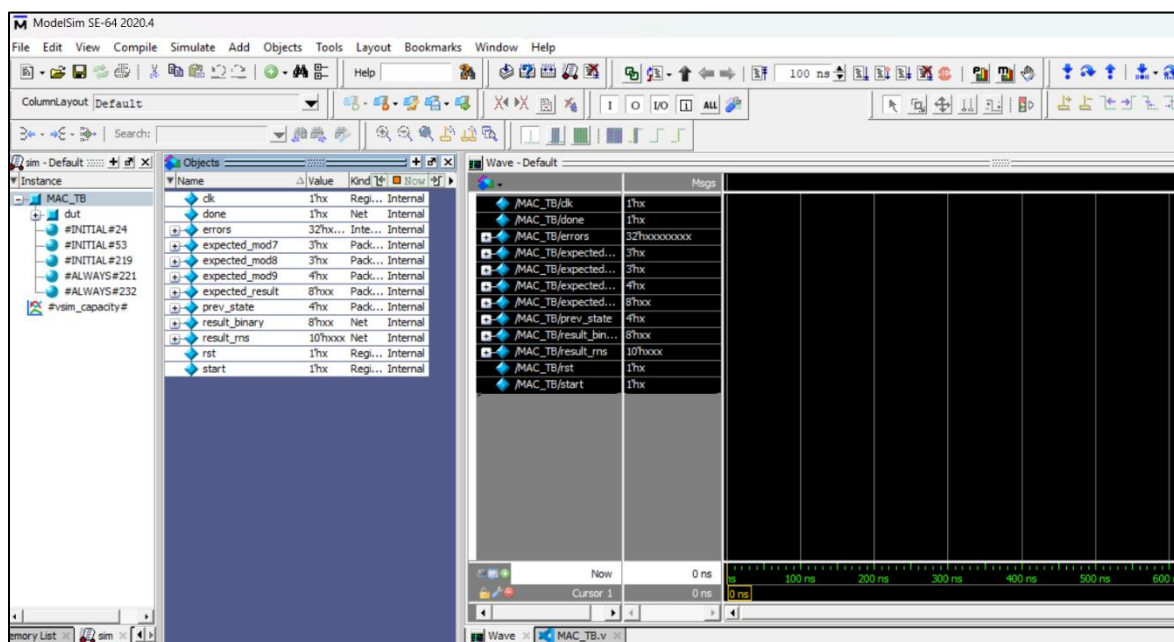
نتیجه مورد انتظار:

$$result = 5 \times 4 + 10 \times 2 + 3 \times 6 = 20 + 20 + 18 = 58$$

نمایش RNS عدد 58:

- $58 \bmod 7 = 2$
- $58 \bmod 8 = 2$
- $58 \bmod 9 = 4$

برای تست دیزاین و صحت سنجی کارکرد کنترلر و دیتایپ نوشته شده برای عملیات MAC در سیستم RNS ابتدا بعد از اینکه روی Run Simulation کلیک کردیم باید گزینه View All Components رو بزنیم که به Sub Module ها و سیگنالهای آنها هم دسترسی داشته باشیم. سپس باید قبل اجرای فایل تست بنچ سیگنالهای آن را به باکس خاکستری بقل آن منتقل کنیم تا بعد از ران گرفتن بتوانیم شکل موج های آن را ببینیم.



شکل ۱۱- اضافه کردن سیگنالها برای مشاهده شکل موج آنها طی فرآیند شبیه سازی

۱.۵.۲ خروجی Testbench

فایل MAC_TB.v به گونه ای نوشته شده است که خود پس از شبیه سازی و اجرا آن خروجی رو توی بخش Transcript و پایین صفحه بنویسد و صحت آن را با مقدار مورد نظر که اینجا ۵۸ هست چک کند.

در صفحه بعدی میتونید خروجی حاصل از اجرا رو مشاهده کنید.

```
#####
#          MAC Unit Testbench - RNS System {7, 8, 9}
#####
```

Test Configuration:

Moduli: {7, 8, 9}

Dynamic Range: $7 * 8 * 9 = 504$

Input Data (from DataMem.txt):

$a[0] = 5, b[0] = 4 \Rightarrow 5 * 4 = 20$

$a[1] = 10, b[1] = 2 \Rightarrow 10 * 2 = 20$

$a[2] = 3, b[2] = 6 \Rightarrow 3 * 6 = 18$

Expected MAC Result: $20 + 20 + 18 = 58$

Expected RNS Representation of 58:

$58 \bmod 7 = 2$

$58 \bmod 8 = 2$

$58 \bmod 9 = 4$

=====

[60] Starting MAC operation...

[90] FSM State: 0 -> 1

[110] FSM State: 1 -> 2

[111] LOAD: $a_int=5, b_int=4$ (sel=0)

[130] FSM State: 2 -> 3

[131] CONVERT: $a_rns=1011010101, b_rns=1001000100$

[150] FSM State: 3 -> 4

[151] MULTIPLY: result=1101000010

[170] FSM State: 4 -> 5

[171] ACCUMULATE: $acc=0000000000 + mult=1101000010 = 1101000010$

...

[350] FSM State: 13 -> 14

=====

RESULTS

=====

RNS Result (10-bit): 0100100100

Bits [9:7] (mod 7): 010 = 2

Bits [6:4] (mod 8): 010 = 2

Bits [3:0] (mod 9): 0100 = 4

Binary Result: 58

Expected: 58


```

=====
                                VERIFICATION
=====

[PASS] mod 7: Got 2, Expected 2
[PASS] mod 8: Got 2, Expected 2
[PASS] mod 9: Got 4, Expected 4
[PASS] Binary: Got 58, Expected 58

=====
                                FINAL STATUS
=====

*****
*      ALL TESTS PASSED!      *
*****

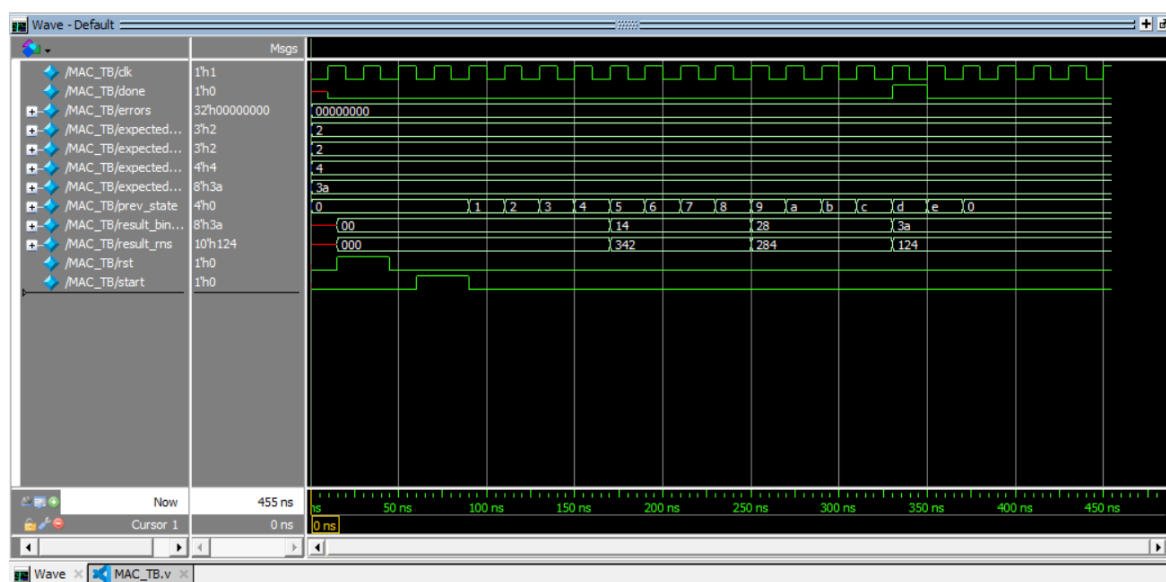
```

خب همان طور که مشاهده میشود تست به درستی اجرا و صحت سنجی شد.

نکته) اگر بخواهیم تست کیس دیگری اجرا کنیم میبایست ابتدا مقادیر ذخیره شده در فایل DataMem.txt رو تغییر دهیم سپس در فایل *MAC_TB.v* تغییرات لازم را انجام دهیم از جمله تغییر مقدار Expected به شکل RNS و Binary.

۱.۵.۳ شکل موجهای خروجی

شکل موجهای شبیه سازی در ModelSim نشان دهنده عملکرد صحیح سیستم است:



شکل ۱۲- شکل موج خروجی حاصل از اجرای تست پنج در نرم افزار مدلسیم

تحلیل شکل موج:

۱. سیگنال `clk`: کلاک سیستم با پریود $20ns$
۲. سیگنال `rst`: ریست در ابتدای شبیه‌سازی فعال و سپس غیرفعال می‌شود
۳. سیگنال `start`: پالس شروع عملیات MAC
۴. سیگنال `prev_state`: نمایش تغییر حالت‌های FSM از ۰ تا ۱۴
۵. سیگنال `result_binary`: مقدار نهایی ۵۸ ($0x3A$) در انتهای عملیات
۶. سیگنال `result_rns`: مقدار RNS نهایی $0x124$ معادل (۲, ۲, ۴)
۷. سیگنال `done`: فعال شدن در حالت DONE

۱.۵.۴ تأیید محاسبات دستی

مرحله ۱: $5 \times 4 = 20$

- $20 \bmod 7 = 6$
- $20 \bmod 8 = 4$
- $20 \bmod 9 = 2$
- RNS: (6, 4, 2)

مرحله ۲: $10 \times 2 = 20$

- RNS: (6, 4, 2)
- Accumulate: $(6 + 6 = 12 \bmod 7 = 5, 4 + 4 = 8 \bmod 8 = 0, 2 + 2 = 4)$
- Result: (5, 0, 4)

مرحله ۳: $3 \times 6 = 18$

- $18 \bmod 7 = 4$
- $18 \bmod 8 = 2$
- $18 \bmod 9 = 0$
- RNS: (4, 2, 0)
- Accumulate: $(5 + 4 = 9 \bmod 7 = 2, 0 + 2 = 2, 4 + 0 = 4)$
- **Final RNS: (2, 2, 4) = 58**

بخش ۲: تحلیل تأخیر و مساحت سخت‌افزاری

اطلاعات پایه از صورت سوال

فرمول‌های تأخیر:

فرمول تأخیر	نوع مدار
$t_d = n \times 30 \text{ ps}$	جمع‌کننده Binary (n بیت)
$t_d = n^2 \times 30 \text{ ps}$	ضرب‌کننده آرایه‌ای Binary (n بیت)
$t_d = (n + 1) \times 30 \text{ ps}$	جمع‌کننده RNS (n بیت)

واحدهای مساحت:

مساحت	جزء
1 GE	Full Adder (FA)
2 GE	CSA
۰ (صرف‌نظر)	MUX

مشخصات سیستم RNS با پیمانه‌های {۷, ۸, ۹}:

محدوده مقادیر	تعداد بیت	پیمانه
۰ تا ۶	۳ بیت	۷
۰ تا ۷	۳ بیت	۸
۰ تا ۸	۴ بیت	۹

محدوده دینامیکی:

$$M = 7 \times 8 \times 9 = 504$$

تعداد بیت معادل در Binary:

$$\lceil \log_2(504) \rceil = 9 \text{ بیت}$$

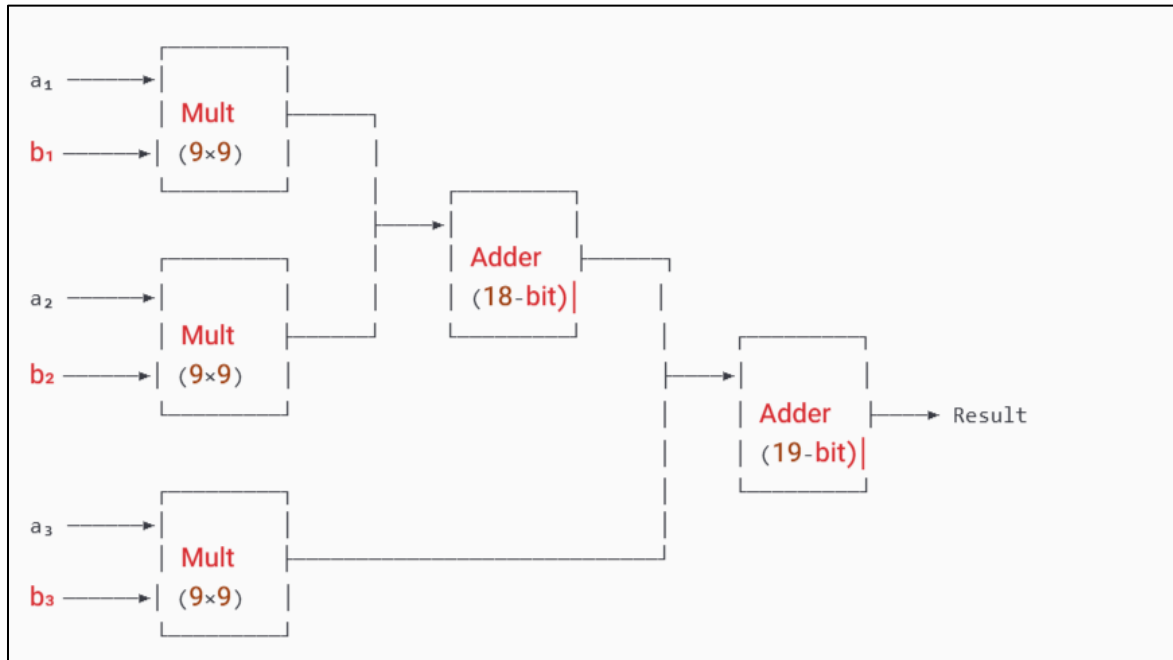
عملیات MAC:

$$result = \sum_{i=1}^3 a_i \cdot b_i = a_1 \cdot b_1 + a_2 \cdot b_2 + a_3 \cdot b_3$$

سوال ۱: محاسبه و مقایسه تأخیر

تحلیل تأخیر در سیستم Binary

ساختار عملیات MAC:



شکل ۱۳- فرمت عملیات Multiply Add در سیستم اعداد باینری

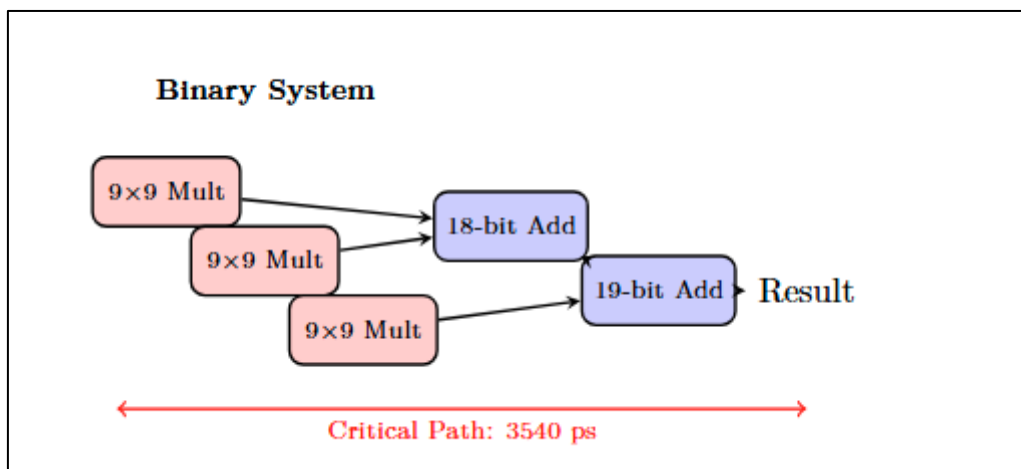
نکته: سه ضرب به صورت موازی انجام می‌شوند، اما دو جمع به صورت سری هستند.

مسیر بحرانی: یک ضرب + دو جمع متوالی

محاسبه تأخیر:

تأخیر	محاسبه	فرمول	عملیات	مرحله
2430 ps	$9^2 \times 30$	$n^2 \times 30$	ضرب ۹×۹ بیت	۱
540 ps	18×30	$n \times 30$	جمع ۱۸ بیت	۲
570 ps	19×30	$n \times 30$	جمع ۱۹ بیت	۳
3540 ps			مجموع	

$$t_{Binary} = 2430 + 540 + 570 = 3540 \text{ ps}$$

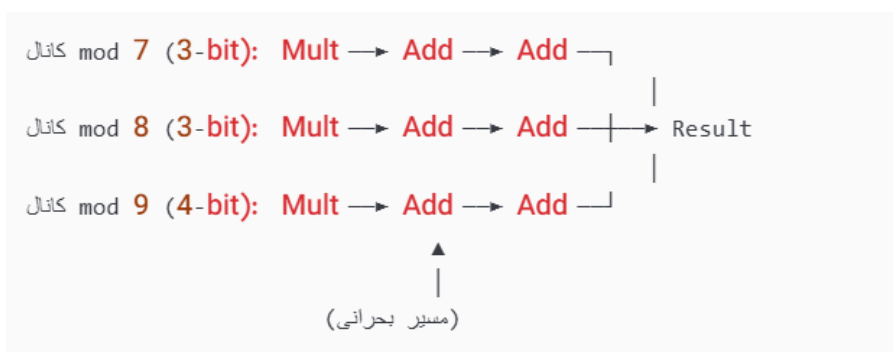


شکل ۱۴- تأخیر عملیات MAC در سیستم باینری

تحلیل تأخیر در سیستم RNS

ساختار عملیات MAC در RNS:

در سیستم RNS، سه کانال (mod 7, mod 8, mod 9) به صورت کاملاً موازی کار می‌کنند.



تأخیر کل = تأخیر کندترین کانال (مسیر بحرانی)

کانال mod 7 (۳ بیتی):

مرحله	عملیات	فرمول	محاسبه	تأخیر
۱	ضرب ۳×۳ بیت	$n^2 \times 30$	$3^2 \times 30$	270 ps
۲	جمع 3 RNS بیت	$(n + 1) \times 30$	$(3 + 1) \times 30$	120 ps
۳	جمع 3 RNS بیت	$(n + 1) \times 30$	$(3 + 1) \times 30$	120 ps

510 ps		مجموع کانال mod 7	
--------	--	-------------------	--

کانال mod 8 (۳ بیتی):

تأخیر	محاسبه	فرمول	عملیات	مرحله
270 ps	$3^2 \times 30$	$n^2 \times 30$	ضرب 3×3 بیت	۱
120 ps	$(3 + 1) \times 30$	$(n + 1) \times 30$	جمع 3 RNS بیت	۲
120 ps	$(3 + 1) \times 30$	$(n + 1) \times 30$	جمع 3 RNS بیت	۳
510 ps			مجموع کانال mod 8	

کانال mod 9 (۴ بیتی) – مسیر بحرانی:

تأخیر	محاسبه	فرمول	عملیات	مرحله
480 ps	$4^2 \times 30$	$n^2 \times 30$	ضرب 4×4 بیت	۱
150 ps	$(4 + 1) \times 30$	$(n + 1) \times 30$	جمع 4 RNS بیت	۲
150 ps	$(4 + 1) \times 30$	$(n + 1) \times 30$	جمع 4 RNS بیت	۳
780 ps			مجموع کانال mod 9	

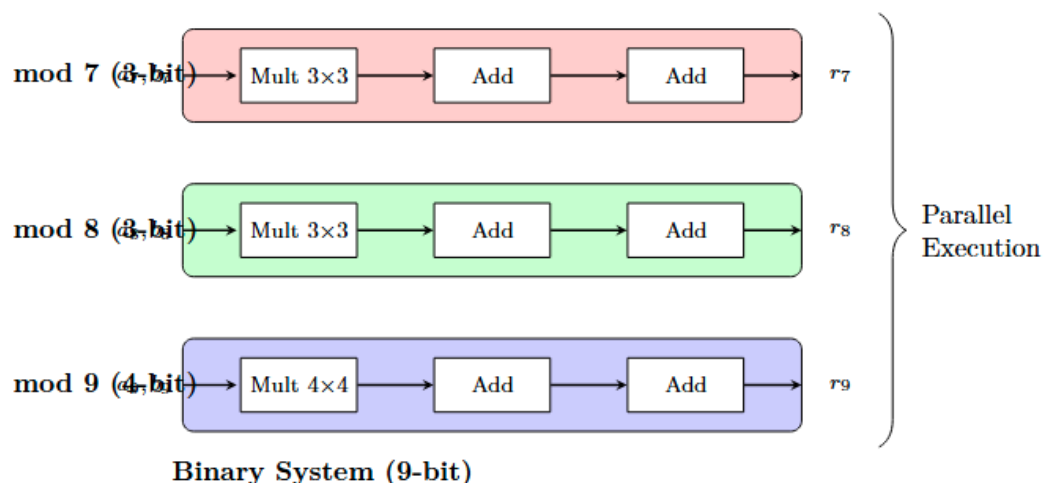
تأخیر کل RNS:

$$t_{RNS} = \max(510, 510, 780) = 780 \text{ ps}$$

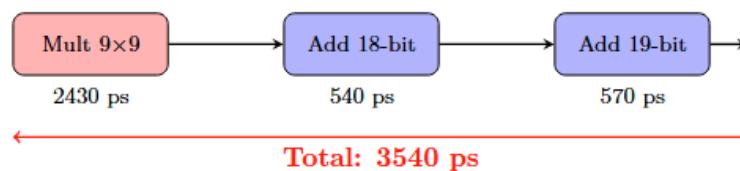
۲.۲.۳ جدول مقایسه تأخیر

نسبت به Binary	تأخیر کل	سیستم
$\times 1/100$ (مرجع)	3540 ps	Binary
$\times 4/54$ سریع‌تر	780 ps	RNS

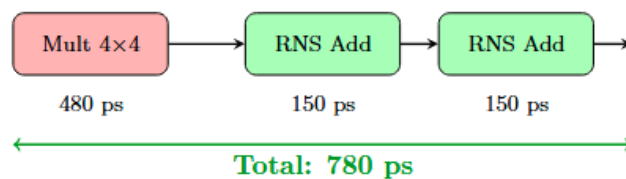
$$\text{نسبت بهبود} = \frac{3540}{780} = 4.54$$



Binary System (9-bit)



RNS System (mod 9 channel - Critical Path)



Speed-up: 4.54×

شکل ۱۵- مقایسه دو سیستم باینری و RNS در تأخیر عملیات MAC

دلایل تفاوت تأخیر

۱. موازی‌سازی ذاتی کانال‌ها: در سیستم RNS، عملیات روی هر پیمانه به صورت مستقل و همزمان انجام می‌شود. تأخیر کل برابر با تأخیر کندترین کانال است، نه مجموع تأخیرها.
۲. حذف انتشار Carry بین کانال‌ها: در سیستم Binary، بیت نقلی (Carry) باید از LSB تا MSB منتقل شود که تأخیر $O(n)$ ایجاد می‌کند. در RNS هیچ وابستگی بین کانال‌ها وجود ندارد.
۳. کاهش پهنای بیت در هر کانال:

• Binary: عملیات روی ۹ بیت

• RNS: عملیات روی ۳ یا ۴ بیت در هر کانال

۴. تأثیر درجه دوم در تأخیر ضرب‌کننده: تأخیر ضرب‌کننده آرایه‌ای $O(n^2)$ است:

- Binary: $9^2 = 81$ واحد تأخیر
- RNS (بدترین حالت): $4^2 = 16$ واحد تأخیر
- کاهش ۵ برابری فقط در ضرب‌کننده

توضیح دلایل تفاوت تأخیر

۱. موازی‌سازی ذاتی کانال‌ها:

در سیستم RNS، عملیات روی هر پیمانه به صورت مستقل و همزمان انجام می‌شود. تأخیر کل برابر با تأخیر کندترین کانال است، نه مجموع تأخیرها. در حالی که در Binary تمام بیت‌ها باید با هم پردازش شوند.

۲. حذف انتشار Carry بین کانال‌ها:

در سیستم Binary، بیت نقلی (Carry) باید از LSB تا MSB منتقل شود که تأخیر خطی $O(n)$ ایجاد می‌کند. در RNS هیچ وابستگی بین کانال‌ها وجود ندارد و Carry فقط درون هر کانال (با بیت‌های کمتر) منتشر می‌شود.

۳. کاهش پهنای بیت در هر کانال:

سیستم	پهنای بیت
Binary	۹ بیت
RNS (mod 7, 8)	۳ بیت
RNS (mod 9)	۴ بیت

۴. تأثیر درجه دوم در تأخیر ضرب‌کننده:

تأخیر ضرب‌کننده آرایه‌ای $O(n^2)$ است. کاهش n تأثیر چشمگیری دارد:

سیستم	محاسبه	واحد تأخیر
Binary (9-bit)	9^2	81
RNS mod 9 (4-bit)	4^2	16
نسبت کاهش		5.06x

سوال ۲: محاسبه مساحت جمع‌کننده RNS

۲.۱ تحلیل کد RNSAdder.v

```

D:\Embedded AI Fall 2025\CA3\CA3\CA3\rns\adder\RNSAdder.v - Default
Ln#
1 module RNSAdder(input [9:0] a , b , input [10:0]mi, output [9:0] c);
2
3 // index 7 to 9 --> radix7
4 // index 4 to 6 --> radix8
5 // index 0 to 3 --> radix9
6
7 wire [3:0] CSA_outoutfor7,CSA_outoutfor8;
8 wire [4:0] CSA_outoutfor9;
9 wire [9:0] adder_output;
10
11 Adder #(.LENGTH(3)) adderfor7 (.a(a[9:7]),.b(b[9:7]),.c(adder_output[9:7]));
12 Adder #(.LENGTH(3)) adderfor8 (.a(a[6:4]),.b(b[6:4]),.c(adder_output[6:4]));
13 Adder #(.LENGTH(4)) adderfor9 (.a(a[3:0]),.b(b[3:0]),.c(adder_output[3:0]));
14 CSA #(.LENGTH(3),.MILENGTH(3)) CSAfor7(.a(a[9:7]),.b(b[9:7]),.mi(mi[10:8]),.c(CSA_outoutfor7));
15 CSA #(.LENGTH(3),.MILENGTH(4)) CSAfor8(.a(a[6:4]),.b(b[6:4]),.mi(mi[7:4]),.c(CSA_outoutfor8));
16 CSA #(.LENGTH(4),.MILENGTH(4)) CSAfor9(.a(a[3:0]),.b(b[3:0]),.mi(mi[3:0]),.c(CSA_outoutfor9));
17
18 assign c[9:7] = CSA_outoutfor7[3]?adder_output[9:7]:CSA_outoutfor7;
19 assign c[6:4] = CSA_outoutfor8[3]?adder_output[6:4]:CSA_outoutfor8;
20 assign c[3:0] = CSA_outoutfor9[4]?adder_output[3:0]:CSA_outoutfor9;
21
22 endmodule
23

```

شکل ۱۶- کد ماژول RNSAdder.v

۲/۲ محاسبه مساحت جمع‌کننده RNS

جمع‌کننده‌های Ripple Carry (Adder):

مساحت	تعداد FA	پهنای بیت	کانال
3 GE	3	۳ بیت	mod 7
3 GE	3	۳ بیت	mod 8
4 GE	4	۴ بیت	mod 9
10 GE	10		جمع جزئی

CSA ها:

مساحت	مساحت واحد	تعداد	کانال
2 GE	2 GE	1	CSA for mod 7
2 GE	2 GE	1	CSA for mod 8
2 GE	2 GE	1	CSA for mod 9
6 GE			جمع جزئی

مساحت کل یک جمع‌کننده RNS:

$$A_{adder,RNS} = 10 + 6 = \boxed{16 GE}$$

۲/۳ محاسبه مساحت جمع‌کننده Binary

مساحت جمع‌کننده 18 بیتی Binary:

$$A_{adder,Binary,18} = 18 \times 1 = 18 GE$$

۲.۴ جدول مقایسه مساحت جمع‌کننده

نسبت	مساحت جمع‌کننده	سیستم
$\times 1/00$ (مرجع)	18 GE	Binary (18-bit)
$\times 0/89$ (کوچک‌تر)	16 GE	RNS

نتیجه: جمع‌کننده RNS با مساحت 16 GE، حدود ۱۱٪ کوچک‌تر از جمع‌کننده 18 بیتی Binary است.

$$\text{نسبت} = \frac{18}{16} = 1.125$$

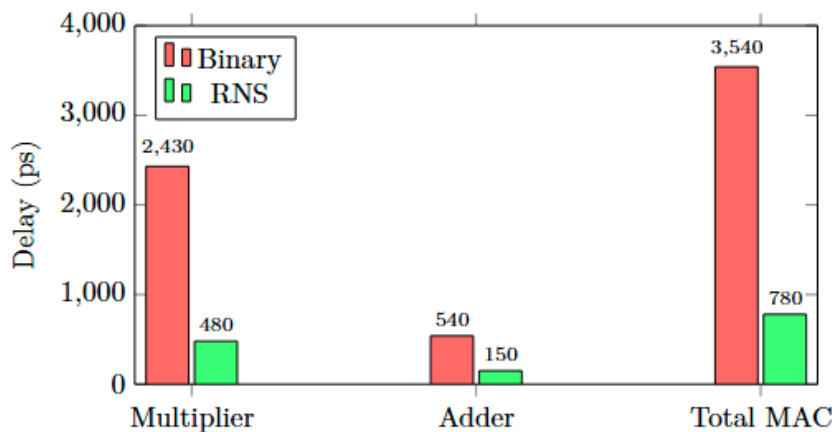
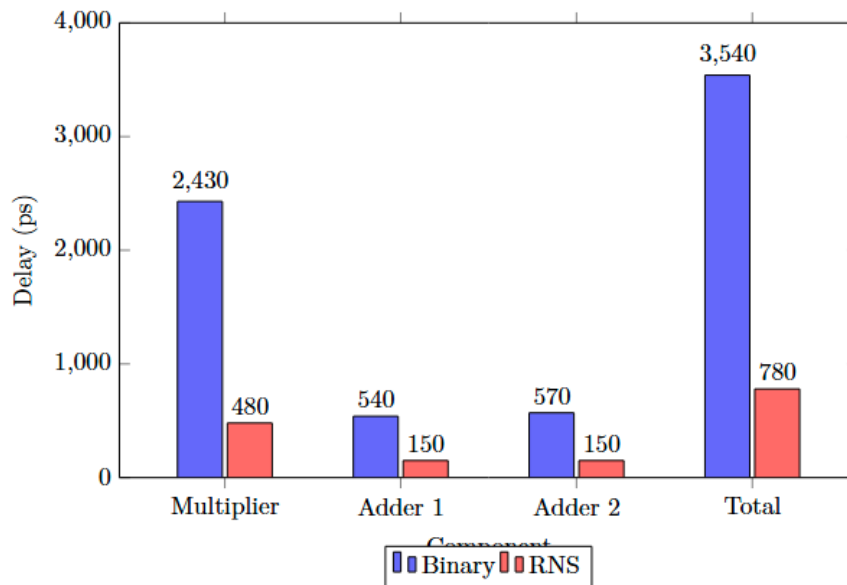


Figure 5: Delay Comparison: Binary vs RNS

شکل ۱۷- مقایسه تاخیر عملیات MAC در دو سیستم به تفکیک اجزا

البته در شکل بالا هر دو adder چون تاخیر مشابه هم دارند با هم نوشته شده اند. شکل دقیق تر در صفحه بعد قابل مشاهده است:



سوال ۳: محاسبه مساحت کل مدار MAC

فرمول مساحت ضرب‌کننده آرایه‌ای

یک ضرب‌کننده آرایه‌ای $n \times n$ بیتی از $n \times (n - 1)$ عدد Full Adder تشکیل شده است:

$$A_{mult}(n) = n \times (n - 1) GE$$

مساحت مدار MAC در سیستم Binary

ضرب‌کننده‌ها: برای MAC به ۳ ضرب‌کننده 9×9 بیتی نیاز داریم:

$$A_{mult,Binary} = 3 \times [9 \times (9 - 1)] = 3 \times 72 = 216 GE$$

جمع‌کننده‌ها:

- جمع‌کننده ۱۸ بیتی: ۱۸ GE
- جمع‌کننده ۱۹ بیتی: ۱۹ GE

$$A_{adder,Binary} = 18 + 19 = 37 GE$$

مساحت کل Binary:

$$A_{total,Binary} = 216 + 37 = \boxed{253 GE}$$

مساحت مدار MAC در سیستم RNS

ضرب‌کننده‌ها: برای MAC به ۳ ضرب‌کننده RNS نیاز داریم. هر ضرب‌کننده RNS شامل سه کانال است:

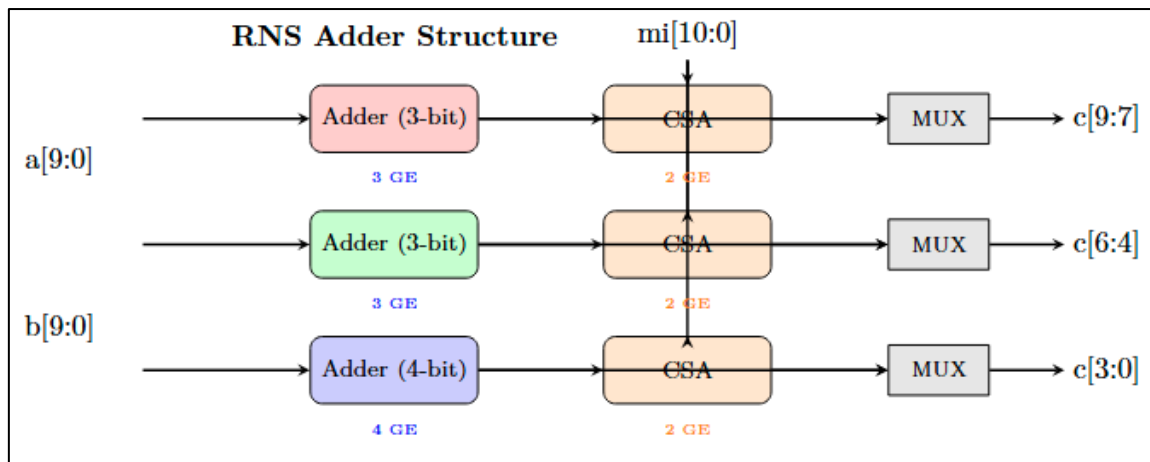
مساحت	تعداد mult	FA per mult	فرمول	اندازه	کانال
18 GE	3	6	3×2	3×3	mod 7
18 GE	3	6	3×2	3×3	mod 8
36 GE	3	12	4×3	4×4	mod 9
72 GE					جمع

جمع‌کننده‌ها: برای MAC به ۲ جمع‌کننده RNS نیاز داریم:

$$A_{adder,RNS} = 2 \times 16 = 32 \text{ GE}$$

مساحت کل RNS:

$$A_{total,RNS} = 72 + 32 = \boxed{104 \text{ GE}}$$



شکل ۱۸- ساختار جمع‌کننده RNS

۲.۴.۴ جدول مقایسه مساحت کل

مقایسه	RNS	Binary	جزء
کوچک‌تر ۳x RNS	72 GE	216 GE	ضرب‌کننده‌ها
کوچک‌تر ۱۴% RNS	32 GE	37 GE	جمع‌کننده‌ها

مجموع	253 GE	104 GE	RNS 2.43x کوچکتر
-------	--------	--------	------------------

$$\text{نسبت کاهش مساحت} = \frac{253}{104} = 2.43$$

در زیر نمودار مقایسه مساحت برای دو سیستم به تفکیک قرار داده شده است:

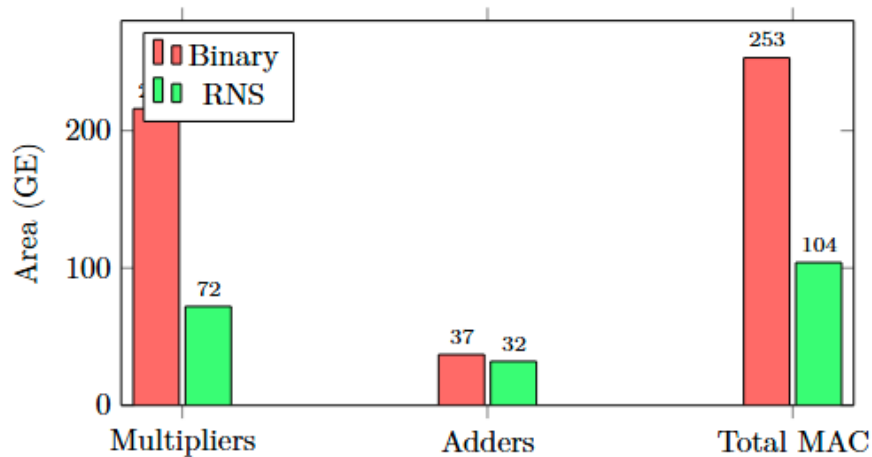


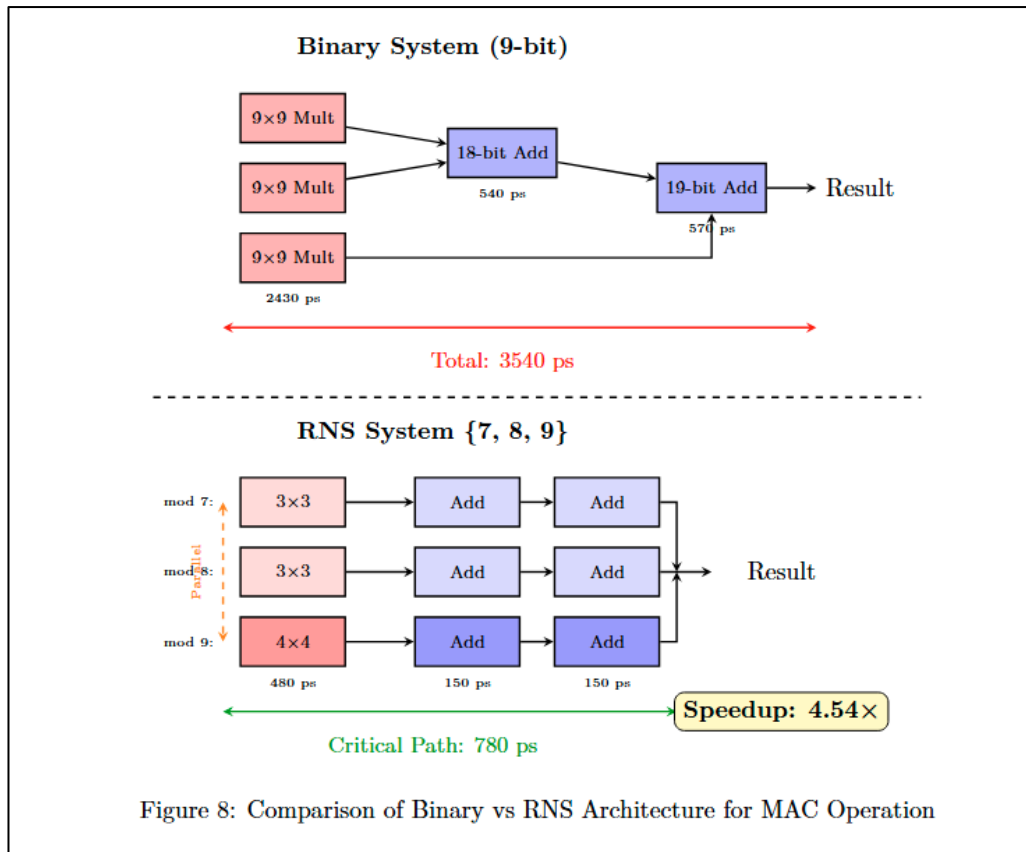
Figure 6: Area Comparison: Binary vs RNS

شکل ۱۹- مقایسه مساحت برای عملیات MAC در دو سیستم

در پایان هم لازم به ذکر است از شماتیک مقایسه دو سیستم از نظر تاخیر در انجام عملیات MAC را بررسی کنیم:

این مورد پیش تر به شکل کامل بررسی شده و صرفاً در اینجا جهت شهودی سازی بهتر ساختارهای MAC در دو سیستم Binary و RNS تصویر قرار داده شده است.

نکته) میتوان در صفحه بعد مشاهده کرد که کاملاً سیستم RNS از موازی سازی جهت کاهش زمان انجام عملیات MAC بهره میبرد و برگ برنده آن نسبت به سیستم باینری است.



جمع‌بندی نهایی

جدول مقایسه کلی Binary و RNS

معیار	Binary	RNS {7,8,9}	بهبود RNS
تأخیر ضرب	2430 ps	480 ps	x0/۰۶ سریع‌تر
تأخیر جمع	540 ps	150 ps	x۳/۶ سریع‌تر
تأخیر کل MAC	3540 ps	780 ps	x۴/۵۴ سریع‌تر
مساحت ضرب‌کننده‌ها	216 GE	72 GE	x۳ کوچک‌تر
مساحت جمع‌کننده‌ها	37 GE	32 GE	x۱۴٪ کوچک‌تر
مساحت کل MAC	253 GE	104 GE	x۲/۴۳ کوچک‌تر

در جدول صفحه بعد همین موارد گفته شده در جدول بالا به شکل تمیز تر آورده شده است:

Table 1: Comparison of RNS and Binary Systems for MAC Operation

Metric	Binary	RNS {7,8,9}	Improvement
Multiplier Delay	2430 ps	480 ps	5.06× faster
Adder Delay	540 ps	150 ps	3.6× faster
Total MAC Delay	3540 ps	780 ps	4.54× faster
Multiplier Area	216 GE	72 GE	3× smaller
Adder Area	37 GE	32 GE	1.12× smaller
Total MAC Area	253 GE	104 GE	2.43× smaller

نتیجه‌گیری

سیستم RNS با پیمانه‌های {۷, ۸, ۹} برای عملیات MAC دارای مزایای قابل توجهی است:

۱. **سرعت بسیار بالاتر:** تأخیر ۴/۵۴ برابر کمتر به دلیل موازی‌سازی کانال‌ها و حذف انتشار Carry
۲. **مساحت کمتر:** مساحت کل ۲/۴۳ برابر کوچک‌تر به دلیل استفاده از ضرب‌کننده‌های با پهنای بیت کمتر
۳. **مصرف توان کمتر:** به دلیل مساحت کمتر و فعالیت سوئیچینگ کمتر
۴. **مناسب برای DSP:** این ویژگی‌ها RNS را برای کاربردهای پردازش سیگنال دیجیتال و فیلترهای دیجیتال بسیار مناسب می‌کند.

((پایان))